

Curso de SQL

Apuntes de contexto y programación

Elaborado y redactado por: Lihúén Villarreal

Índice de contenidos

Notas de contexto.....	4
SQL.....	4
Lenguaje declarativo.....	4
Dialectos de SQL.....	4
DDL y DML.....	4
Bases de datos relacionales.....	5
Claves primarias (PRIMARY KEY).....	5
Claves foráneas (FOREIGN KEY).....	5
Tipos de relaciones entre tablas.....	6
Programación con SQL.....	6
Consultas (query).....	6
SELECT (selección de columnas).....	6
FROM (origen de los datos).....	7
AS (alias de tablas y columnas).....	7
Orden lógico de ejecución de una query.....	8
Comentarios en SQL (--, /* */).....	8
Filtrar datos.....	9
WHERE (selección de filas).....	9
Operadores de comparación (=, <>, <, >, <=, >=).....	9
Operadores lógicos (AND, OR, NOT).....	9
IN.....	10
BETWEEN (rangos).....	10
LIKE (patrones de texto) y comodines (% , _).....	10
DISTINCT.....	10
IS NULL / IS NOT NULL.....	11
Lógica ternaria.....	11
Ordenar, limitar y agrupar datos.....	11
ORDER BY (ASC, DESC).....	11
LIMIT y OFFSET.....	12
GROUP BY.....	12
HAVING.....	12
Diferencia entre WHERE y HAVING.....	13
Funciones y agregación de datos.....	13
Funciones integradas (built-in functions).....	13
Agregación de datos.....	14
NULL y funciones de agregación.....	14

Conteos con DISTINCT.....	15
Ventanas (OVER / PARTITION BY).....	15
NULLIF (reemplazar por NULL).....	15
COALESCE (reemplazar NULL).....	16
Joins (combinación de tablas).....	17
Buenas prácticas de nombres y claves.....	17
JOIN.....	17
INNER JOIN.....	17
LEFT JOIN.....	18
RIGHT JOIN.....	18
FULL JOIN (OUTER JOIN).....	19
JOIN implícito vs JOIN explícito.....	19
Joins con múltiples tablas.....	19
Operar con conjuntos.....	20
UNION / UNION ALL.....	20
Reglas del UNION y UNION ALL.....	20
Diferencia entre UNION y JOIN.....	20
Subconsultas.....	20
Subconsultas en WHERE.....	21
Subconsultas en FROM.....	21
Subconsultas en SELECT.....	21
Subconsultas correlacionadas.....	22
Condicionales y evaluaciones.....	22
EXISTS / NOT EXISTS (evaluación).....	22
CASE WHEN (condicionales).....	23
Ejemplo combinado.....	23
DDL (definición y modificación de estructura de datos).....	24
Tipos de datos más comunes.....	24
Restricciones básicas (constraints).....	24
PRIMARY KEY.....	24
FOREIGN KEY.....	24
UNIQUE.....	24
NOT NULL.....	25
DEFAULT.....	25
CHECK.....	25
CREATE TABLE.....	25
ALTER TABLE.....	25
DROP TABLE.....	26
TRUNCATE TABLE.....	26
Diferencias entre DELETE, TRUNCATE y DROP.....	26
DML (modificación de datos).....	26
Carga de datos en tablas (load tables).....	26
INSERT.....	27
UPDATE.....	27
ON UPDATE.....	27
DELETE.....	28
ON DELETE.....	28
Uso cuidadoso de WHERE en operaciones destructivas.....	28

Vistas e índices.....	29
VIEW (vistas).....	29
INDEX (índices).....	29
Índices compuestos (multicolumn index).....	29
Impacto de los índices en performance.....	30
Seguridad y permisos.....	30
Usuarios y roles.....	30
GRANT.....	31
REVOKE.....	31
Metadatos (información sobre la base).....	31
Information schema.....	31
Nombres comunes en information_schema.....	32
Consultar el information_schema.....	33
Obtener información de tablas.....	33
Obtener información de columnas.....	33
Transacciones.....	34
SAVEPOINT (ROLLBACK parcial).....	34
Principios ACID.....	35
Performance y buenas prácticas.....	35
Legibilidad y mantenimiento de queries.....	36
EXPLAIN / EXPLAIN ANALYZE.....	36
Patrones comunes de consulta.....	36
Buscar duplicados.....	36
Buscar faltantes.....	37
Top N por grupo.....	37
Fechas más recientes.....	37
Conteos condicionados.....	38
Acceder a bases de datos desde otros lenguajes.....	38
Driver (conector).....	38
SQL desde Python.....	38

Notas de contexto

SQL

- **SQL (Structured Query Language)** es un lenguaje de programación **interpretado** (no requiere compilar) y **declarativo**.
- Se utiliza para interactuar con **bases de datos relacionales**. Permite consultar, insertar, modificar y eliminar datos, así como definir y modificar la estructura de la base.

Lenguaje declarativo

- Un lenguaje **declarativo** es aquel en el que solo se **declara qué resultado se espera obtener**, pero no se indican las instrucciones o pasos del procedimiento para obtener esos resultados (como se hace en los lenguajes **imperativos**).

Dialectos de SQL

SQL es un **estándar**, pero cada motor implementa variaciones (**dialectos**).

Los más comunes:

- PostgreSQL
- MySQL / MariaDB
- SQLite
- IBM Db2
- SQL Server

Las diferencias suelen estar en:

- Tipos de datos
- Funciones disponibles
- Sintaxis específica

La mayor parte del **SQL básico es compartido** entre dialectos.

DDL y DML

Son **tipos de sentencias SQL**.

- **DDL (Data Definition Language):**
Se usa para definir o modificar la estructura de la base de datos.
 - Ejemplos:
 - CREATE TABLE
 - ALTER TABLE
 - DROP TABLE

→ **DML (Data Manipulation Language):**

Se usa para trabajar con los datos dentro de las tablas.

- Ejemplos:

SELECT
INSERT
UPDATE
DELETE

Bases de datos relacionales

Una base de datos **relacional** organiza la información en **tablas**, formadas por:

- **Filas (registros):** cada instancia de datos
- **Columnas (campos):** cada atributo del dato

Las tablas se relacionan entre sí mediante **claves**. Esto es una característica central del modelo relacional: los datos no se duplican, sino que se conectan mediante las claves.

Ejemplo conceptual:

- Tabla **usuarios**
- Tabla **pedidos**
- Cada pedido pertenece a un usuario, pero la información del usuario se guarda una sola vez.

Claves primarias (PRIMARY KEY)

→ Una **clave primaria** identifica de forma **única** cada **fila** de una tabla.

Características:

- No se repite
- No puede ser **NULL**
- Suele ser un **ID** numérico

Ejemplo:

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY,  
  nombre TEXT,  
  email TEXT  
);
```

Claves foráneas (FOREIGN KEY)

→ Una **clave foránea** es una columna que referencia la clave primaria de otra tabla. Se usa para **relacionar tablas**.

Ejemplo:

```
CREATE TABLE pedidos (  
    id INTEGER PRIMARY KEY,  
    usuario_id INTEGER,  
    total DECIMAL,  
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id)  
);
```

Tipos de relaciones entre tablas

- **1-1:** Una fila de una tabla se relaciona con una sola fila de otra tabla.
 - Ejemplo: Una persona ↔ Una cédula de identidad.
- **1-N:** Una fila de una tabla se relaciona con varias filas de otra tabla.
 - Ejemplo: Un cliente ↔ Varios pedidos.
- **N-N:** Varias filas de una tabla se relacionan con varias filas de otra tabla.
 - En bases de datos relacionales no se implementa directamente, sino mediante una **tabla intermedia** (puente).
 - Ejemplo: Estudiantes ↔ Cursos (Tabla intermedia: Inscripciones)

Programación con SQL

Consultas (query)

- Una **query (consulta)** es una **instrucción** SQL que se envía a la base de datos para **obtener o manipular información**.
- En el uso cotidiano, el término suele referirse especialmente a consultas de datos con **SELECT**.

SELECT (selección de columnas)

- **SELECT** es una **cláusula** SQL que se utiliza para seleccionar **columnas** de una tabla (base de datos).
 - Ejemplo:

```
SELECT nombre, email  
FROM usuarios;
```

 - ↪ Retorna todos los valores de las columnas **nombre** y **email** de todos los usuarios.

- Con ***** retorna **todas** las columnas de la tabla (es decir, las filas completas):

```
SELECT *  
FROM usuarios;
```

→ En **SELECT** se pueden usar **expresiones**, no solo columnas simples.

→ Las expresiones pueden incluir:

- Operaciones matemáticas.
- Concatenaciones.
- Funciones integradas.

Ejemplo de **SELECT** con expresión y función:

```
SELECT nombre, ROUND(salario * 1.1, 2)  
FROM empleados;
```

↪ Retorna los nombres de los empleados, y sus salarios con un 10% de aumento, redondeados con dos decimales.

Nota: En SQL, las **sentencias** siempre se **finalizan con punto y coma (;)**.

FROM (origen de los datos)

→ **FROM** indica **de qué tabla o tablas** provienen los datos que se están consultando.

→ Sin **FROM**, no se pueden consultar tablas (salvo en casos muy puntuales según el dialecto).

Ejemplo:

```
SELECT nombre  
FROM clientes;
```

AS (alias de tablas y columnas)

→ **AS** permite asignar un **alias** (nombre alternativo temporal) a una **columna o tabla** dentro de una query.

- No modifica la base de datos ni guarda ningún dato.
- El uso de alias mejora la legibilidad y es clave al trabajar con múltiples tablas.

Alias de **columnas**:

```
SELECT nombre, salario * 1.1 AS salario_con_aumento  
FROM empleados;
```

↪ Asigna el alias **salario_con_aumento** a la columna **salario * 1.1** retornada.

Alias de **tablas**:

```
SELECT u.nombre  
FROM usuarios AS u;
```

↪ Asigna el alias **u** a la tabla **usuarios** y luego la utiliza con el alias en el **SELECT**.

Nota: Para el caso de **alias de tablas**, el escribir el **AS** es **opcional**.

Orden lógico de ejecución de una query

→ Aunque las cláusulas de una query se **escriben** en un orden, el motor las **ejecuta** en otro orden.

→ Esto explica por qué, por ejemplo, un alias definido en **SELECT** no puede usarse en **WHERE**.

Orden lógico simplificado:

1. **FROM**
2. **WHERE**
3. **GROUP BY**
4. **HAVING**
5. **SELECT**
6. **ORDER BY**
7. **LIMIT**

Comentarios en SQL (--, /* */)

→ Los comentarios sirven para **documentar queries** o desactivar partes del código.

Comentario de **una sola línea**:

```
-- Este es un comentario  
SELECT * FROM usuarios;
```

Comentario de **múltiples líneas**:

```
/*  
Este es el comentario:  
Consulta de usuarios activos  
*/  
SELECT *  
FROM usuarios;
```


Filtrar datos

WHERE (selección de filas)

→ La cláusula **WHERE** se utiliza para seleccionar **filas** según una **condición**.

Ejemplo:

```
SELECT nombre, email
FROM usuarios
WHERE activo = true;
```

↪ Sin **WHERE**, la query retorna **todas** las filas de las columnas seleccionadas.
Con **WHERE**, retorna solo los valores de las columnas **nombre** y **email** en cuyas filas el atributo **activo** sea **true**.

Operadores de comparación (=, <>, <, >, <=, >=)

→ Se usan para **comparar valores** en una **condición**.

Operadores más comunes:

=	igual
<>	distinto
<	menor que
>	mayor que
<=	menor o igual
>=	mayor o igual

Ejemplo:

```
SELECT *
FROM productos
WHERE precio >= 100;
```

Operadores lógicos (AND, OR, NOT)

→ Permiten **combinar o negar condiciones**.

- **AND**: todas las condiciones deben cumplirse.
- **OR**: al menos una condición debe cumplirse.
- **NOT**: niega una condición.

Ejemplo:

```
SELECT *
```

```
FROM usuarios
WHERE activo = true AND edad >= 18;
```

IN

- **IN** permite verificar **si un valor pertenece a una lista** de valores.
- Es una alternativa más legible que múltiples **OR**.

Ejemplo:

```
SELECT *
FROM pedidos
WHERE estado IN ('pendiente', 'enviado');
```

BETWEEN (rangos)

- **BETWEEN** se usa para filtrar **valores dentro de un rango incluyendo los extremos**.
- Puede usarse con números, fechas y textos.

Ejemplo:

```
SELECT *
FROM productos
WHERE precio BETWEEN 100 AND 500;
```

LIKE (patrones de texto) y comodines (% , _)

- **LIKE** se usa para buscar **patrones de texto**.

Comodines:

- % representa cualquier cantidad de caracteres
- _ representa un solo carácter

Ejemplo:

```
SELECT *
FROM usuarios
WHERE email LIKE '%@gmail.com';
```

- ↪ Retorna las filas completas de los usuarios cuyo email termina en **@gmail.com**.

DISTINCT

- **DISTINCT** **elimina filas duplicadas** en el resultado.

Ejemplo:

```
SELECT DISTINCT pais
FROM usuarios;
```

- ↪ Retorna la lista de países sin repetir valores.
Aplica al conjunto completo de columnas seleccionadas.

IS NULL / IS NOT NULL

- Se usan para verificar valores **nulos** .
 - **NULL** no se puede comparar con **=** o **<>** .

Ejemplo:

```
SELECT *
FROM usuarios
WHERE telefono IS NULL;
```

Lógica ternaria

- **NULL** representa la **ausencia de valor** .
 - No es lo mismo que **0** , una cadena vacía (**' '**) ni **FALSE** .
 - En SQL, **NULL** significa **UNKNOWN** : "desconocido" o "no aplicable".

SQL no usa lógica booleana binaria (verdadero / falso), sino **lógica ternaria** :

- **TRUE**
- **FALSE**
- **UNKNOWN**

- Cualquier comparación con **NULL** devuelve **UNKNOWN** .

Ejemplo:

```
SELECT *
FROM usuarios
WHERE email = NULL;
```

- ↪ No retorna resultados, porque la comparación es **UNKNOWN** .

Ordenar, limitar y agrupar datos

ORDER BY (ASC, DESC)

- **ORDER BY** se utiliza para **ordenar** el resultado de una query (no reordena la base de datos) **según una o más columnas** .

→ Órdenes posibles:

- **ASC**: orden **ascendente** (menor a mayor, $A \rightarrow Z$)
Este es el orden por default y puede quedar implícito en la consulta.
- **DESC**: orden **descendente** (mayor a menor, $Z \rightarrow A$)
Esta opción siempre hay que escribirla explícitamente.

Ejemplo:

```
SELECT nombre, fecha_alta
FROM usuarios
ORDER BY fecha_alta DESC;
```

LIMIT y OFFSET

→ **LIMIT** restringe la **cantidad de filas** devueltas.

→ **OFFSET** indica desde qué fila empezar.

Ejemplo:

```
SELECT *
FROM productos
ORDER BY precio
LIMIT 10 OFFSET 20;
```

↪ Fórmula muy usada para **paginación** de resultados.

GROUP BY

→ **GROUP BY** se utiliza para **agrupar filas** y aplicar **funciones de agregación** sobre cada grupo.

- **Regla**: Toda columna en el **SELECT** debe estar en una función de agregación o en el **GROUP BY**.

Ejemplo:

```
SELECT pais, COUNT(*)
FROM usuarios
GROUP BY pais;
```

↪ Retorna un conteo de cuántos usuarios hay por cada país.

HAVING

→ **HAVING** se usa para **filtrar grupos**, después de aplicar agregaciones.

Ejemplo:

```
SELECT pais, COUNT(*)
FROM usuarios
GROUP BY pais
HAVING COUNT(*) > 100;
```

↪ Retorna solo los países que tienen más de 100 usuarios.

Diferencia entre **WHERE** y **HAVING**

→ **WHERE** filtra **filas individuales**, antes de agrupar.

→ **HAVING** filtra **grupos**, después de agrupar.

Ejemplo combinado:

```
SELECT pais, COUNT(*)
FROM usuarios
WHERE activo = true
GROUP BY pais
HAVING COUNT(*) > 100;
```

↪ Retorna los países con más de 100 usuarios activos.

Funciones y agregación de datos

Funciones integradas (built-in functions)

- Las **funciones integradas** son funciones que provee el motor de la base de datos para operar sobre valores (texto, números, fechas, etc.).
- Se las llama solo con el nombre de la función, y los **parámetros** entre paréntesis, separados por comas, cuando corresponda.

Funciones comunes de SQL:

Función	Descripción breve
<code>COUNT(*)</code>	Cuenta la cantidad de filas
<code>COUNT(columna)</code>	Cuenta valores no nulos de la columna
<code>SUM(columna)</code>	Suma valores numéricos
<code>AVG(columna)</code>	Calcula el promedio
<code>MIN(columna)</code>	Retorna el valor mínimo
<code>MAX(columna)</code>	Retorna el valor máximo

UPPER(string)	Convierte a mayúsculas
LOWER(string)	Convierte a minúsculas
TRIM(string)	Elimina espacios al inicio y al final
LENGTH(string)	Devuelve la longitud del texto
SUBSTRING(string, inicio, longitud)	Extrae parte de un texto
ROUND(numero, decimales)	Redondea un número
ABS(numero)	Retorna el valor absoluto
CEILING(numero)	Redondea hacia arriba
FLOOR(numero)	Redondea hacia abajo
CURRENT_DATE	Devuelve la fecha actual
CURRENT_TIME	Devuelve la hora actual
CURRENT_TIMESTAMP	Devuelve fecha y hora actual

Agregación de datos

- La **agregación de datos** en SQL consiste en **resumir múltiples filas en un único valor** (o en un conjunto reducido de valores), aplicando funciones que calculan totales, promedios, mínimos, máximos o conteos.
- Las **funciones de agregación** operan sobre **conjuntos de filas** y retornan un **único valor por grupo** (o para toda la tabla). Las principales son:
 - **COUNT** → Cuenta valores no nulos de una columna, o cantidad de filas de una tabla (*).
 - **SUM** → Suma valores numéricos.
 - **AVG** → Calcula el promedio.
 - **MIN** → Retorna el valor mínimo.
 - **MAX** → Retorna el valor máximo.
- La agregación suele usarse junto con **GROUP BY** para obtener resúmenes por grupo.
 - Ejemplo:


```
SELECT departamento, AVG(salario)
FROM empleados
GROUP BY departamento;
```

 ↪ Retorna el salario promedio por departamento.

NULL y funciones de agregación

- **COUNT(*)** cuenta **todas** las filas.
- **COUNT(columna)** **ignora** valores **NULL**.

Conteos con DISTINCT

- **DISTINCT** puede combinarse con funciones de agregación para **evitar contar valores repetidos**.

Ejemplo:

```
SELECT COUNT(DISTINCT pais)
FROM usuarios;
```

- ↪ Retorna la cantidad de países distintos presentes en la tabla **usuarios**.

Ventanas (OVER / PARTITION BY)

- Una **función de ventana** (*window function*) es una función que **calcula un valor usando varias filas**.
- **No agrupa** esas filas en una sola (como **GROUP BY** y las funciones de agregación), sino que muestra el resultado **en cada fila**.
- Utiliza:
 - **OVER** para **definir** la ventana (indicar sobre qué filas se ejecuta la función).
 - **PARTITION BY** (opcional) para **dividir** las filas en grupos lógicos pero sin agruparlas (es como decir: “*Calculá esto por cada grupo, pero mostralo en cada fila*”).

Ejemplo:

```
SELECT nombre, departamento, salario,
       RANK() OVER (
         PARTITION BY departamento
         ORDER BY salario DESC
       ) AS ranking
FROM empleados;
```

- ↪ Retorna el ranking de salarios dentro de cada departamento, sin perder filas.

NULLIF (reemplazar por NULL)

- **NULLIF(a, b)** es una función escalar integrada de SQL que retorna:
 - **NULL** si **a=b**.
 - **a** si **a≠b**.
- Se usa para:
 - Convertir valores en **NULL**.
 - Tratar valores “sentinela” como si no fueran valores reales.

- Evitar errores, como **divisiones por cero**.
- Evitar que ciertos valores entren en agregaciones.

Ejemplos de aplicación:

```
SELECT nombre, NULLIF(telefono, '')
FROM clientes;
```

⇒ Si `telefono` es un string vacío (''), retorna `NULL` en esa fila.

```
SELECT ventas / NULLIF(cantidad, 0)
FROM productos;
```

⇒ Si `cantidad = 0` → `NULLIF` y la división retornan `NULL` → No hay error.
Si `cantidad ≠ 0` → División normal.

COALESCE (reemplazar NULL)

→ `COALESCE` es una función escalar integrada de SQL que retorna **el primer valor no NULL** de una **lista de expresiones**.

- Si todas son `NULL`, retorna `NULL`.

→ Se usa para:

- Configurar **valores por defecto**.
- Evitar `NULL` en resultados.
- Combinar columnas que pueden tener faltantes.
- Priorizar una fuente sobre otra.

Ejemplos de aplicación:

```
SELECT nombre, COALESCE(telefono, 'Sin teléfono')
FROM clientes;
```

⇒ Si `telefono` es `NULL`, retorna 'Sin teléfono' en esa fila.

```
SELECT precio - COALESCE(descuento, 0)
FROM productos;
```

⇒ Sin `COALESCE`, si `descuento` es `NULL`, el cálculo retorna `NULL`.
Con `COALESCE`, si `descuento` es `NULL`, el cálculo retorna `precio`.

```
SELECT nombre,
       COALESCE(email_personal, email_laboral, email_alternativo)
FROM usuarios;
```

⇒ Retorna el email personal; si no hay, el laboral; si no hay, el alternativo.

```
SELECT *
FROM empleados
WHERE COALESCE(fecha_baja, CURRENT_DATE) >= CURRENT_DATE;
```

⇒ Trata el `NULL` como "sigue activo".

Uso combinado con `NULLIF`:

```
COALESCE(NULLIF(valor, 0), 1)
```

→ Si `valor` es `0`, `NULLIF` lo convierte en `NULL`, y luego `COALESCE` lo convierte en `1`.

Joins (combinación de tablas)

Buenas prácticas de nombres y claves

→ Al trabajar con múltiples tablas, es importante mantener criterios claros y consistentes.

Buenas prácticas comunes:

- Usar nombres de tablas en plural (`usuarios`, `pedidos`).
- Usar un campo `id` como clave primaria.
- Nombrar claves foráneas como `<tabla>_id`.
- Mantener el mismo tipo de dato entre clave primaria y foránea.

Ejemplo:

```
usuario_id INTEGER REFERENCES usuarios(id)
```

→ Facilita la lectura, el mantenimiento y el uso de joins.

JOIN

→ Un `JOIN` permite **combinar filas de dos o más tablas relacionadas**, basándose en una condición que normalmente vincula claves (por ejemplo, una clave primaria con una clave foránea).

- Un `JOIN` responde preguntas que requieren información distribuida en distintas tablas.
- Ejemplo conceptual: obtener datos de usuarios junto con datos de sus pedidos.

INNER JOIN

→ Retorna **solo las filas que tienen coincidencia en ambas tablas** según la condición del `JOIN`.

Sintaxis típica:

```
SELECT columnas  
FROM tabla_a  
INNER JOIN tabla_b ON condicion;
```

Ejemplo:

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios
INNER JOIN pedidos ON usuarios.id = pedidos.usuario_id;
```

↪ Retorna solo los usuarios que tienen pedidos, junto con la fecha de cada pedido.

LEFT JOIN

→ Retorna **todas las filas de la tabla izquierda** y las **coincidencias de la tabla derecha**. Si no hay coincidencia, los valores de la tabla derecha aparecen como **NULL**.

Sintaxis típica:

```
SELECT columnas
FROM tabla_a
LEFT JOIN tabla_b ON condicion;
```

Ejemplo:

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios
LEFT JOIN pedidos ON usuarios.id = pedidos.usuario_id;
```

↪ Retorna todos los usuarios, tengan o no pedidos, y los que no tienen pedidos muestran **NULL** en la fecha de pedido.

RIGHT JOIN

→ Retorna **todas las filas de la tabla derecha** y las **coincidencias de la tabla izquierda**. Si no hay coincidencia, los valores de la tabla izquierda aparecen como **NULL**.

★ **Nota:** En la práctica, se recomienda no usar **RIGHT JOIN**, sino reescribirlo como **LEFT JOIN**, invirtiendo el orden de las tablas, para mejorar la legibilidad.

Sintaxis típica:

```
SELECT columnas
FROM tabla_a
RIGHT JOIN tabla_b ON condicion;
```

Ejemplo:

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios
RIGHT JOIN pedidos ON usuarios.id = pedidos.usuario_id;
```

↪ Retorna todos los pedidos, incluso si alguno no tiene un usuario asociado.

FULL JOIN (OUTER JOIN)

- Retorna **todas las filas de ambas tablas**, haya o no coincidencia. Cuando no hay match, los valores faltantes aparecen como **NULL**.

Sintaxis típica:

```
SELECT columnas
FROM tabla_a
FULL JOIN tabla_b ON condicion;
```

Ejemplo:

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios
FULL JOIN pedidos ON usuarios.id = pedidos.usuario_id;
```

- ↪ Retorna todos los usuarios y pedidos, incluso los que no tienen correspondencia entre sí.

JOIN implícito vs JOIN explícito

- Ambos hacen lo mismo, pero el **JOIN explícito** es más legible, menos propenso a errores, y estándar en SQL moderno.

JOIN implícito (no recomendado):

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios, pedidos
WHERE usuarios.id = pedidos.usuario_id;
```

JOIN explícito (recomendado):

```
SELECT usuarios.nombre, pedidos.fecha
FROM usuarios
INNER JOIN pedidos ON usuarios.id = pedidos.usuario_id;
```

Joins con múltiples tablas

- Se pueden encadenar varios **JOIN** para combinar más de dos tablas.

Ejemplo:

```
SELECT usuarios.nombre, pedidos.id, productos.nombre
FROM usuarios
INNER JOIN pedidos ON usuarios.id = pedidos.usuario_id
INNER JOIN productos ON pedidos.producto_id = productos.id;
```

- ↪ Combina información de usuarios, pedidos y productos en una sola consulta.

Operar con conjuntos

→ Operar con conjuntos implica **combinar los resultados de dos o más queries** en un solo resultado, siempre que tengan una estructura compatible.

UNION / UNION ALL

→ **UNION** combina los resultados de dos queries **eliminando filas duplicadas**.

○ Sintaxis:

```
SELECT columna1, columna2
FROM tabla1
UNION
SELECT columna1, columna2
FROM tabla2;
```

↪ Retorna un conjunto de filas únicas provenientes de ambas consultas.

→ **UNION ALL** combina los resultados de dos queries **conservando los duplicados**.

○ Sintaxis:

```
SELECT columna1, columna2
FROM tabla1
UNION ALL
SELECT columna1, columna2
FROM tabla2;
```

↪ Retorna todas las filas de ambas consultas, incluso si se repiten.

Reglas del UNION y UNION ALL

- Las queries deben tener **el mismo número de columnas**.
- Las columnas deben ser de **tipos compatibles**.
- Los nombres de columnas del resultado final los toma del **primer SELECT**.
- **ORDER BY** solo puede ir **al final**, después del último **SELECT**.

Diferencia entre UNION y JOIN

UNION

- Combina resultados verticalmente
- Apila filas
- No relaciona filas entre sí
- Requiere estructura compatible

JOIN

- Combina tablas horizontalmente
- Agrega columnas
- Relaciona filas mediante claves
- Requiere condición de unión

Subconsultas

→ Una **subconsulta** (*subquery*) es una consulta SQL que está contenida **dentro** de otra consulta. Se aplica como **SELECT anidados** (*nested SELECT*).

- Se usan para obtener un valor o conjunto de valores que utiliza la consulta principal (la consulta externa depende del resultado de la interna.).
- Sintácticamente, siempre va **entre paréntesis**.
- Pueden retornar:
 - Un valor único (escalares).
 - Una columna.
 - Varias filas.

Subconsultas en **WHERE**

- Se usan para filtrar resultados según datos calculados en otra consulta.

Ejemplo:

```
SELECT nombre
FROM empleados
WHERE salario > (
  SELECT AVG(salario)
  FROM empleados
);
```

- ↪ Retorna los empleados que ganan más que el salario promedio.

Subconsultas en **FROM**

- La subconsulta actúa como una **tabla temporal**.
- **Siempre** se debe asignar un **alias** a la tabla temporal generada por la subconsulta en el **FROM**.

Ejemplo:

```
SELECT depto, AVG(salario) AS salario_promedio
FROM (
  SELECT departamento AS depto, salario
  FROM empleados
) AS tabla_temp
GROUP BY depto;
```

- ↪ La subconsulta se ejecuta primero y crea una tabla temporal **tabla_temp**. La consulta calcula el salario promedio por departamento usando **tabla_temp**.

Subconsultas en **SELECT**

- Se usan para calcular valores fila por fila.

Ejemplo:

```

SELECT nombre,
      (
        SELECT COUNT(*)
        FROM pedidos p
        WHERE p.usuario_id = u.id
      ) AS total_pedidos
FROM usuarios u;

```

↪ Muestra cada usuario junto con la cantidad de pedidos que hizo.

Subconsultas correlacionadas

→ Son subconsultas que **dependen de la fila actual** de la consulta externa. Se ejecutan una vez por cada fila.

Ejemplo:

```

SELECT nombre
FROM empleados e
WHERE salario > (
  SELECT AVG(salario)
  FROM empleados
  WHERE departamento = e.departamento
);

```

↪ Retorna los empleados que ganan más que el promedio de su propio departamento.

Condicionales y evaluaciones

EXISTS / NOT EXISTS (evaluación)

→ Se usan para **evaluar** si una subconsulta devuelve filas. No importa el contenido, solo si hay resultados.

Ejemplo con EXISTS:

```

SELECT nombre
FROM usuarios u
WHERE EXISTS (
  SELECT 1
  FROM pedidos p
  WHERE p.usuario_id = u.id
);

```

↪ Retorna los usuarios que tienen al menos un pedido.

Ejemplo con NOT EXISTS:

```

SELECT nombre
FROM usuarios u
WHERE NOT EXISTS (
    SELECT 1
    FROM pedidos p
    WHERE p.usuario_id = u.id
);

```

↪ Retorna los usuarios que no tienen pedidos.

CASE WHEN (condicionales)

→ CASE WHEN permite aplicar **lógica condicional** dentro de una query.

Forma típica:

```

CASE
    WHEN condicion THEN valor
    ELSE valor
END

```

Ejemplo:

```

SELECT nombre,
CASE
    WHEN salario >= 50000 THEN 'alto'
    ELSE 'bajo'
END AS nivel_salario
FROM empleados;

```

↪ Clasifica a los empleados según su salario.

Ejemplo combinado

```

SELECT
    nombre,
CASE
    WHEN EXISTS (
        SELECT 1
        FROM pedidos p
        WHERE p.id_cliente = c.id
    )
    THEN 'con pedidos'
    ELSE 'sin pedidos'
END AS estado

```

```
FROM clientes c;
```

↪ Retorna a los clientes clasificados según si tienen pedidos o no.

DDL (definición y modificación de estructura de datos)

→ **DDL (Data Definition Language)** agrupa las sentencias que se usan para **crear, modificar o eliminar la estructura** de la base de datos: tablas, columnas y restricciones. No trabaja sobre los datos en sí, sino sobre su forma.

Tipos de datos más comunes

Los tipos de datos pueden variar según el **dialecto**, pero estos son los más habituales:

- **INT, INTEGER**: números enteros
- **DECIMAL, NUMERIC**: números con decimales
- **VARCHAR(n)**: texto de longitud variable
- **CHAR(n)**: texto de longitud fija
- **TEXT**: texto largo
- **DATE**: fecha
- **TIME**: hora
- **TIMESTAMP / DATETIME**: fecha y hora
- **BOOLEAN**: verdadero / falso

Restricciones básicas (constraints)

- La **integridad** de los datos significa que la información almacenada en la base de datos sea **correcta, consistente y confiable** a lo largo del tiempo.
- Las **restricciones (constraints)** en SQL definen reglas de integridad que se aplican a columnas o tablas para:
 - Limitar los valores permitidos.
 - Evitar datos inválidos.

PRIMARY KEY

```
id INT PRIMARY KEY
```

- Identifica de forma única cada fila.
- Garantiza **unicidad y no nulidad**.

FOREIGN KEY

```
FOREIGN KEY (cliente_id) REFERENCES clientes(id)
```

- Define una relación con otra tabla.
- Garantiza **integridad referencial** (asegura que las relaciones entre tablas se mantengan consistentes, impide que una fila apunte a un registro inexistente).

UNIQUE

```
email VARCHAR(150) UNIQUE
```

- Impide valores duplicados en una columna.
- Garantiza **unicidad**.

NOT NULL

```
nombre VARCHAR(100) NOT NULL
```

- Impide valores nulos.
- Garantiza **no nulidad**.

DEFAULT

```
activo BOOLEAN DEFAULT true
```

- Define un valor **por defecto**.
- Garantiza un valor automático si no se especifica otro.

CHECK

```
edad INT CHECK (edad >= 0)
```

- Permite **definir** restricciones con condiciones **personalizadas**.
- Se evalúa al insertar o actualizar filas.

CREATE TABLE

- **Crea una nueva tabla** y define sus columnas y restricciones.

Ejemplo:

```
CREATE TABLE usuarios (  
  id INT PRIMARY KEY,  
  nombre VARCHAR(100),  
  email VARCHAR(150)  
);
```

- ↪ Crea la tabla `usuarios` con sus columnas y estructura inicial.

ALTER TABLE

- **Modifica la estructura** de una tabla existente.

Ejemplos:

```
ALTER TABLE usuarios
```

```
ADD fecha_alta DATE;
```

↪ **Agrega** una columna nueva a la tabla.

```
ALTER TABLE usuarios
```

```
ALTER COLUMN nombre TYPE VARCHAR(150);
```

↪ **Cambia** el tipo de dato de una columna existente.

DROP TABLE

→ **Elimina completamente** una tabla.

Ejemplo:

```
DROP TABLE usuarios;
```

↪ Borra la tabla y todos sus datos.

TRUNCATE TABLE

→ **Elimina** todos los **datos** de una tabla, pero **conserva su estructura**.

Ejemplo:

```
TRUNCATE TABLE usuarios;
```

↪ Vacía la tabla de forma rápida.

Diferencias entre DELETE, TRUNCATE y DROP

- **DELETE** borra datos específicos.
- **TRUNCATE** vacía la tabla.
- **DROP** elimina la tabla.

Sentencia	Qué elimina	Mantiene estructura	Usa WHERE
DELETE	Filas	Sí	Sí
TRUNCATE	Todas las filas	Sí	No
DROP	Tabla completa	No	No

DML (modificación de datos)

→ **DML (*Data Manipulation Language*)** agrupa las sentencias que se usan para **insertar, modificar y eliminar datos** dentro de las tablas. A diferencia de DDL, trabaja sobre el contenido, no sobre la estructura.

Carga de datos en tablas (load tables)

→ Carga datos desde **archivos externos** (CSV, TXT, etc.).

★ La sintaxis depende del motor.

Ejemplo:

```
LOAD DATA INFILE 'usuarios.csv'
INTO TABLE usuarios;
```

↪ Importa datos desde un archivo a la tabla.

Nota: Esta sintaxis no es **estándar** SQL, existen formas alternativas según el motor (**COPY** en PostgreSQL, etc.).

INSERT

→ **Inserta nuevas filas** en una tabla.

Ejemplo:

```
INSERT INTO usuarios (nombre, email)
VALUES ('Ana', 'ana@mail.com');
```

↪ Agrega una nueva fila con esos valores.

```
INSERT INTO usuarios (nombre, email)
VALUES
    ('Juan', 'juan@mail.com'),
    ('María', 'maria@mail.com');
```

↪ Inserta **varias filas** en una sola sentencia.

UPDATE

→ **Modifica valores** existentes en una tabla.

Ejemplo:

```
UPDATE usuarios
SET activo = false
WHERE id = 5;
```

↪ Cambia el valor de la columna **usuarios** en filas específicas.

ON UPDATE

→ Define qué ocurre cuando se actualiza una clave primaria referenciada.

→ **Opciones** comunes:

- **CASCADE**: Actualiza las claves relacionadas.
- **SET NULL**: Pone la **FOREIGN KEY** en **NULL**.
- **RESTRICT / NO ACTION**: Impide la actualización.

Ejemplo:

```
FOREIGN KEY (usuario_id)
REFERENCES usuarios(id)
ON UPDATE CASCADE
```

DELETE

→ **Elimina filas** de una tabla.

Ejemplo:

```
DELETE FROM usuarios
WHERE activo = false;
```

↪ Borra las filas que cumplen la condición.

ON DELETE

→ Define qué ocurre cuando se elimina una fila referenciada.

→ **Opciones** comunes:

- **CASCADE**: Elimina también las filas relacionadas.
- **SET NULL**: Pone la **FOREIGN KEY** en **NULL**.
- **RESTRICT / NO ACTION**: Impide el borrado.

Ejemplo:

```
FOREIGN KEY (usuario_id)
REFERENCES usuarios(id)
ON DELETE CASCADE
```

Uso cuidadoso de WHERE en operaciones destructivas

→ **UPDATE** y **DELETE** sin **WHERE** **afectan a todas las filas**.

Ejemplo:

```
DELETE FROM usuarios;
```

↪ Elimina todos los registros de la tabla.

Buenas prácticas:

- Probar primero con **SELECT**.
- Usar **WHERE** siempre que sea posible.
- En bases con transacciones, usar **BEGIN** y **ROLLBACK** para verificar cambios.

Vistas e índices

VIEW (vistas)

- Una **vista** es una consulta guardada que se comporta como una tabla virtual.
 - No almacena datos propios (en general), sino que muestra datos de una o más tablas subyacentes.
- Las vistas se usan principalmente para:
 - Simplificar queries complejas.
 - Reutilizar lógica de consulta.
 - Mejorar legibilidad.
 - Limitar acceso a ciertas columnas o filas.

Ejemplo:

```
CREATE VIEW vista_usuarios_activos AS
SELECT id, nombre, email
FROM usuarios
WHERE activo = true;
```

- ↪ Crea una vista que muestra solo los usuarios activos.

```
SELECT *
FROM vista_usuarios_activos;
```

- ↪ Consulta la vista como si fuera una tabla.

INDEX (índices)

- Un **índice** es una estructura auxiliar que acelera la búsqueda de datos en una tabla.

Ejemplo:

```
CREATE INDEX idx_usuarios_email
ON usuarios(email);
```

- ↪ Crea un índice sobre la columna **email**.

Índices compuestos (multicolumn index)

- Incluyen más de una columna.
- Se usan para acelerar consultas que combinan **filtrar** por una o más columnas con **ordenar o agrupar** por otra columna.
- El **orden** es importante: las consultas solo pueden filtrar usando los índices en el orden en que son definidos (de izquierda a derecha).

Ejemplo:

```
CREATE INDEX idx_empleados_salario_ingreso
```

```
ON empleados (salario, fecha_ingreso);
```

↪ Crea el índice compuesto.

```
SELECT *  
FROM empleados  
WHERE salario >= 50000  
ORDER BY fecha_ingreso;
```

↪ Filtra los empleados por **salario** y los ordena por **fecha_ingreso**.

Nota: Esto no sería posible si el índice compuesto se hubiese definido como **(fecha_ingreso, salario)**.

Impacto de los índices en performance

→ Ventajas:

- Aceleran consultas con **WHERE**, **JOIN**, **ORDER BY**, **GROUP BY**.
- Mejoran el rendimiento en tablas grandes.

→ Costos:

- Ocupan espacio extra.
- Ralentizan **INSERT**, **UPDATE** y **DELETE**.

→ Reglas prácticas:

- Indexar columnas usadas frecuentemente en filtros y joins.
- Evitar índices innecesarios.
- No indexar columnas con muy pocos valores distintos (baja cardinalidad).

Seguridad y permisos

→ La seguridad en SQL se refiere al control de **quién puede acceder a la base de datos y qué acciones puede realizar**.

→ Se gestiona principalmente mediante **usuarios, roles y permisos**.

Buenas prácticas de seguridad:

- Otorgar **el menor privilegio necesario**.
- Usar roles en lugar de permisos directos a usuarios.
- Preferir vistas para exponer **datos sensibles**.
- Evitar usar usuarios con permisos administrativos para tareas comunes.

Usuarios y roles

- **Usuario:** entidad que se conecta a la base de datos.
- **Rol:** conjunto de permisos que puede asignarse a uno o varios usuarios.

→ Los roles permiten **gestionar permisos** de forma más simple y mantenible.

Ejemplo:

```
CREATE ROLE analista;
```

↪ Crea un rol que luego puede recibir permisos.

GRANT

→ **GRANT** se usa para **otorgar permisos** a roles o usuarios sobre objetos de la base de datos.

Ejemplo:

```
GRANT SELECT, INSERT  
ON usuarios  
TO analista;
```

↪ Permite al rol **analista** leer e insertar datos en la tabla **usuarios**.

```
GRANT SELECT  
ON vista_usuarios_activos  
TO juan;
```

↪ Permite al usuario **juan** consultar la vista.

REVOKE

→ **REVOKE** se usa para **retirar permisos previamente otorgados**.

Ejemplo:

```
REVOKE INSERT  
ON usuarios  
FROM analista;
```

↪ Quita el permiso de inserción sobre la tabla.

Metadatos (información sobre la base)

→ Los **metadatos** son “datos sobre los datos”.

- Describen la **estructura de la base de datos**: qué tablas existen, qué columnas tienen, qué tipos de datos usan y qué restricciones aplican.
- Se usan para explorar, entender y documentar una base de datos **sin modificarla**.

Information schema

→ El **information_schema** es un conjunto **estándar** de **tablas** que describen la estructura de una base de datos.

→ Incluye información sobre:

- Tablas.
- Columnas.
- Tipos de datos.
- Claves primarias y foráneas.
- Restricciones.
- Vistas.
- Permisos.

→ Se usa para:

- Explorar bases de datos ajenas.
- Entender esquemas grandes o heredados.
- Verificar restricciones y relaciones.
- Automatizar documentación o validaciones.

Nombres comunes en `information_schema`

Nombre	Significado
<code>information_schema</code>	Contenedor estándar de metadatos del sistema
<code>information_schema.tables</code>	Lista de todas las tablas y vistas visibles
<code>information_schema.columns</code>	Lista de todas las columnas de todas las tablas
<code>information_schema.views</code>	Información específica sobre vistas
<code>information_schema.schemata</code>	Lista de esquemas existentes
<code>information_schema.table_constraints</code>	Restricciones definidas sobre tablas
<code>information_schema.key_column_usage</code>	Qué columnas participan en claves o constraints
<code>information_schema.referential_constraints</code>	Detalles de claves foráneas
<code>table_schema</code>	Esquema al que pertenece la tabla (<code>public</code> , etc.)
<code>table_name</code>	Nombre de la tabla o vista
<code>table_type</code>	Tipo: <code>BASE TABLE</code> o <code>VIEW</code>
<code>column_name</code>	Nombre de la columna
<code>constraint_name</code>	Nombre de la restricción
<code>data_type</code>	Tipo de dato lógico (<code>integer</code> , etc.)
<code>character_maximum_length</code>	Longitud máxima (para texto)
<code>numeric_precision</code>	Precisión numérica
<code>numeric_scale</code>	Cantidad de decimales
<code>column_default</code>	Valor por defecto

<code>is_nullable</code>	Si permite <code>NULL</code> (<code>YES / NO</code>)
<code>ordinal_position</code>	Posición de la columna en la tabla
<code>constraint_type</code>	Tipo: <code>PRIMARY KEY</code> , <code>FOREIGN KEY</code> , <code>UNIQUE</code> , <code>CHECK</code>
<code>unique_constraint_name</code>	Constraint referenciada por una <code>FOREIGN KEY</code>
<code>update_rule</code>	Regla ON UPDATE de una <code>FOREIGN KEY</code>
<code>delete_rule</code>	Regla ON DELETE de una <code>FOREIGN KEY</code>
<code>view_definition</code>	SQL que define la vista
<code>is_updatable</code>	Si la vista permite UPDATE
<code>grantee</code>	Usuario o rol que recibe el permiso
<code>privilege_type</code>	Tipo de permiso (<code>SELECT</code> , <code>INSERT</code> , etc.)
<code>is_grantable</code>	Si puede volver a otorgarse

Consultar el `information_schema`

Ejemplo de sintaxis:

```
SELECT *
FROM information_schema.table_constraints
WHERE table_name = 'usuarios';
```

↪ Retorna las restricciones definidas sobre esa tabla.

Obtener información de tablas

→ Permite listar las tablas existentes en una base de datos o esquema.

Ejemplo:

```
SELECT table_name
FROM information_schema.tables
WHERE table_schema = 'public';
```

↪ Retorna el listado de tablas disponibles en ese esquema.

Nota: La forma exacta puede variar según el motor (`SHOW TABLES`, `\dt`, etc.).

Obtener información de columnas

→ Permite ver las columnas de una tabla, sus tipos de datos y propiedades.

Ejemplo:

```
SELECT column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_name = 'usuarios';
```

↪ Retorna los detalles estructurales de las columnas de la tabla.

Transacciones

- Una **transacción** es un **conjunto de operaciones** SQL que se ejecutan como una unidad lógica **indivisible**. O se ejecutan todas correctamente, o no se aplica **ninguna**.
- Se usan para mantener la **consistencia de los datos**, especialmente en **operaciones críticas**.

Elementos de la transacción:

- **BEGIN** → **Inicia** la transacción: marca el inicio de un bloque de operaciones que se pueden confirmar o deshacer.
- **COMMIT** → **Confirma** definitivamente los cambios realizados en la transacción: **guarda** los cambios en la base de datos.
- **ROLLBACK** → **Revierde** todos los cambios realizados desde el último **BEGIN**: **deshace** los cambios y deja la base de datos como estaba antes.
Nota: **ROLLBACK** solo tiene validez después de un **BEGIN** y antes del **COMMIT**.

Ejemplo:

```
BEGIN;
```

```
UPDATE cuentas  
SET saldo = saldo - 100  
WHERE id = 1;
```

```
UPDATE cuentas  
SET saldo = saldo + 100  
WHERE id = 2;
```

```
COMMIT;
```

- ↔ Transfiere dinero entre cuentas de forma segura.
Si se escribía **ROLLBACK** en lugar del **COMMIT**, la transacción no se ejecutaba.

SAVEPOINT (ROLLBACK parcial)

- **SAVEPOINT** permite **retroceder solo hasta cierto punto** de la transacción.

Ejemplo:

```
BEGIN;
```

```
UPDATE cuentas  
SET saldo = saldo - 100
```

```
WHERE id = 1;

SAVEPOINT antes_de_sumar;

UPDATE cuentas
SET saldo = saldo + 100
WHERE id = 2;

ROLLBACK TO antes_de_sumar;

COMMIT;
```

- ⇒ En este caso, el descuento de la primera cuenta se mantiene, pero la suma de la segunda cuenta se revierte.

Principios ACID

Las transacciones cumplen cuatro **principios** que garantizan la **confiabilidad y seguridad** en **operaciones críticas**:

- **Atomicidad**: todo o nada.
- **Consistencia**: los datos quedan en un estado válido.
- **Aislamiento**: las transacciones no interfieren entre sí.
- **Durabilidad**: los cambios confirmados persisten.

Performance y buenas prácticas

- La **performance** (rendimiento) en SQL se refiere a la **rapidez y eficiencia** con la que una consulta retorna resultados, especialmente cuando trabaja con **grandes volúmenes de datos**.
- Las **buenas prácticas** buscan mejorar el rendimiento conservando la **claridad y mantenibilidad**.

Algunos ejemplos:

- Evitar **SELECT ***, consultar solo las **columnas necesarias** para evitar mayor consumo de memoria.
- Aplicar los filtros **WHERE** lo antes posible, así se reduce la cantidad de filas procesadas.
- Evitar **JOINs** innecesarios.
- Unir tablas sobre columnas **indexadas** mejora el rendimiento.
- Indexar columnas usadas en **WHERE** y **JOIN**.

- No abusar de los índices.

Legibilidad y mantenimiento de queries

→ Una query legible suele ser más fácil de mantener y optimizar.

Buenas prácticas:

- Usar alias claros.
- Formatear el SQL de manera consistente.
- Separar líneas por cláusula.
- Comentar queries complejas.

EXPLAIN / EXPLAIN ANALYZE

→ **EXPLAIN** muestra cómo el motor **planea** ejecutar una query.

- Sintaxis:

```
EXPLAIN
SELECT *
FROM tabla
```

→ **EXPLAIN ANALYZE** **ejecuta** la query y muestra el **plan real con tiempos**.

- Sintaxis:

```
EXPLAIN ANALYZE
SELECT *
FROM tabla
```

→ Sirven para:

- Detectar queries lentas.
- Ver uso de índices.
- Entender decisiones del optimizador.

→ La **salida** depende mucho del **motor** de base de datos.

Patrones comunes de consulta

Algunos patrones frecuentes de queries “típicas” de análisis de datos:

- Conteos por categoría.
- Promedios por período.
- Comparación entre grupos.
- Evolución temporal.

Buscar duplicados

→ Se usa para encontrar valores repetidos en una columna.

Ejemplo:

```
SELECT email, COUNT(*)
FROM usuarios
GROUP BY email
HAVING COUNT(*) > 1;
```

↪ Retorna los emails que aparecen más de una vez.

Buscar faltantes

→ Se hace con `LEFT JOIN + IS NULL` (**anti-join**).

Ejemplo:

```
SELECT u.nombre
FROM usuarios u
LEFT JOIN pedidos p ON u.id = p.usuario_id
WHERE p.id IS NULL;
```

↪ Retorna los nombres de los usuarios que no tienen pedidos.

Top N por grupo

→ Se usa para obtener los mejores o peores registros dentro de cada grupo.

Ejemplo:

```
SELECT departamento, nombre, salario
FROM empleados e
WHERE salario = (
    SELECT MAX(salario)
    FROM empleados
    WHERE departamento = e.departamento
);
```

↪ Retorna el empleado mejor pago de cada departamento.

Fechas más recientes

→ Se usa para obtener el último registro por entidad.

Ejemplo:

```
SELECT usuario_id, MAX(fecha)
FROM pedidos
GROUP BY usuario_id;
```

↪ Retorna la fecha del pedido más reciente por usuario.

Conteos condicionados

→ Se usa para contar solo las filas que cumplen una condición.

Ejemplo:

```
SELECT
    COUNT(*) AS total,
    COUNT(CASE WHEN activo = true THEN 1 END) AS activos
FROM usuarios;
```

↪ Cuenta usuarios totales y cuántos están activos.

Acceder a bases de datos desde otros lenguajes

→ Es posible ejecutar sentencias SQL **desde otro lenguaje de programación** (por ejemplo, Python) para consultar o modificar datos de una base de datos.

→ SQL se encarga de las consultas; el lenguaje **anfitrión** se encarga de la lógica, el flujo y el uso de los resultados.

Driver (conector)

→ Un **driver** o **conector** es una **biblioteca** que permite a un lenguaje comunicarse con una base de datos específica.

→ El driver:

- Abre la conexión.
- Envía queries SQL.
- Recibe resultados.

Ejemplos comunes:

- `psycopg2` / `psycopg` → PostgreSQL
- `sqlite3` → SQLite
- `mysql-connector` → MySQL
- `pyodbc` → SQL Server / Db2

SQL desde Python

→ Python es uno de los lenguajes más usados para trabajar con bases de datos, análisis de datos y automatización.

El flujo general es:

1. Conectarse a la base de datos.
2. Ejecutar queries SQL.
3. Obtener resultados.

4. Usar esos resultados en el programa.

Ejemplo:

```
import sqlite3

# Conexión a la base de datos
conn = sqlite3.connect("ejemplo.db")
cursor = conn.cursor()

# Ejecutar una query
cursor.execute("SELECT nombre, email FROM usuarios")

# Obtener resultados
resultados = cursor.fetchall()

# Usar los datos
for nombre, email in resultados:
    print(nombre, email)

# Cerrar conexión
conn.close()
```